



•OPERATING SYSTEM

•INSY 8223

Faculty of IT, GROUP D&F

Dr Eric NIZEYIMANA and Mr. Archimede IRAKOZE

eric.nizeyimana@auca.ac.rw | +250 783 308 544

Lesson 5. Multithreading and Deadlocks

What is a Thread?

A **thread** is the smallest unit of CPU execution within a process.

Threads in the same process share:

- Memory and resources
- Code and data

Each thread has its own:

- Program counter
- Registers
- Stack

Example: Web browser with threads for UI, loading, and rendering.

Multithreading in Operating System

A **thread** is a path that is followed during a program's execution.

The majority of programs written nowadays run as a single thread.

For example, a program is not capable of reading keystrokes while making drawings. These tasks cannot be executed by the program at the same time. This problem can be solved through multitasking so that two or more tasks can be executed simultaneously.

What is Multithreading?

Multithreading is a feature in operating systems that allows a program (Process) to do several tasks at the same time.

Multithreading makes your computer work better by using its resources more effectively, leading to quicker and smoother performance for applications like web browsers, games, and many other programs you use every day.

How Does Multithreading Work?

Multithreading works by allowing a computer's processor to handle multiple tasks at the same time. Even though the processor can only do one thing at a time, it switches between different threads from various programs so quickly that it looks like everything is happening all at once.

What is Multitasking?

Multitasking is the ability of an operating system to run multiple programs or tasks at the same time. It allows you to perform different activities simultaneously on your computer. For example, you can listen to music while browsing the internet and typing a document all at once.

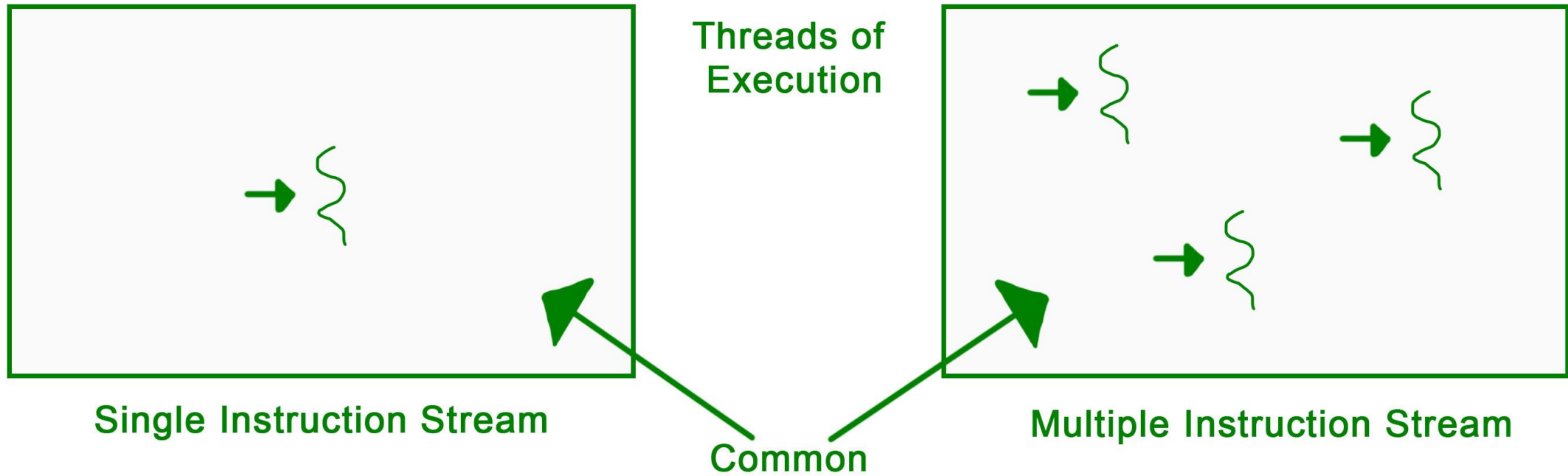
Multitasking types

Multitasking is of two types: Processor-based and thread-based. ***Processor-based multitasking is managed by the OS***, however, multitasking through ***multithreading can be controlled by the programmer to some extent***.

The concept of **multi-threading** needs a proper understanding of these two terms – **a process and a thread**. A process is a program being executed. A process can be further divided into independent units known as threads. A thread is like a small light-weight process within a process. Or we can say a collection of threads is what is known as a process.

Single-threaded Process

Multi-threaded Process



Single Thread and Multi Thread Process

Multithreading vs Multitasking

Feature	Multithreading	Multitasking
Definition	Running multiple threads within a single program simultaneously.	Running multiple programs(Processes) or tasks concurrently.
Example	Web browser loading a page, handling user input, and downloading files simultaneously.	Listening to music, browsing the web, and typing a document at the same time.
Scope	Within a single program.	Across multiple programs.
Resource Use	Utilizes CPU resources more efficiently within a program.	Manages system resources to allocate time and memory to different programs.
Purpose	Enhances the performance and responsiveness of a single application.	Improves overall system efficiency by allowing concurrent execution of multiple programs.
Switching	Threads are managed by the program itself.	Programs are managed by the operating system, which switches between them.

Lifecycle of a Thread

There are various stages in the lifecycle of a thread. Following are the stages a thread goes through in its whole life.

- **New:** The lifecycle of a born thread (new thread) starts in this state. It remains in this state till a program starts.
- **Runnable :** A thread becomes runnable after it starts. It is considered to be executing the task given to it.
- **Waiting :** While waiting for another thread to perform a task, the currently running thread goes into the waiting state and then transitions back again after receiving a signal from the other thread.
- **Timed Waiting:** A runnable thread enters into this state for a specific time interval and then transitions back when the time interval expires or the event the thread was waiting for occurs.
- **Terminated (Dead) :** A thread enters into this state after completing its task.

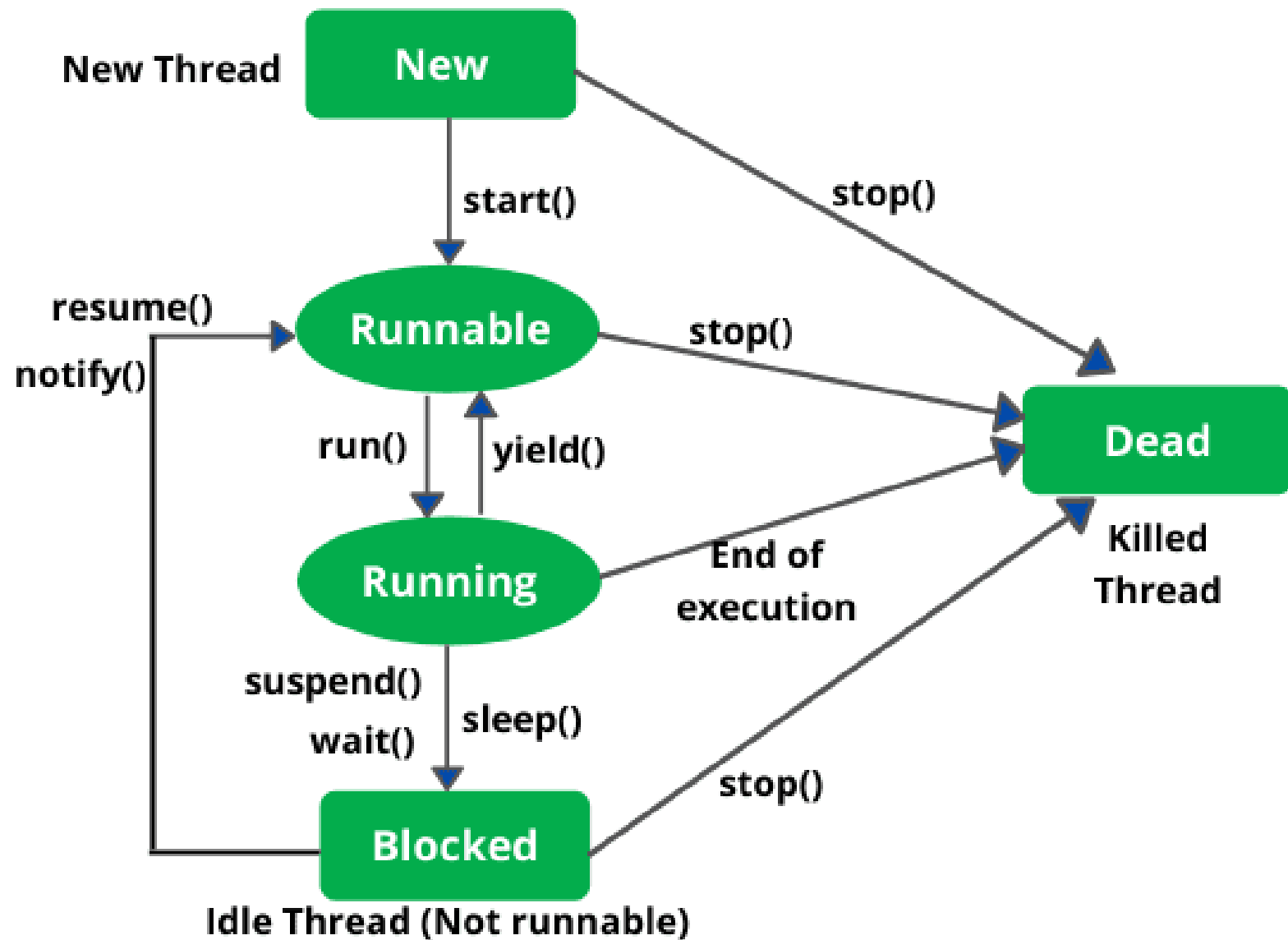


Fig: State Transition Diagram of a Thread

Types of Execution in OS

There are two types of execution:

- **Concurrent Execution:** This occurs when a processor is successful in switching resources between threads in a multithreaded process on a single processor.
- **Parallel Execution:** This occurs when every thread in the process runs on a separate processor at the same time and in the same multithreaded process

Drawbacks of Multithreading

Multithreading is complex and many times difficult to handle. It has a few drawbacks. These are:

- If you don't make use of the locking mechanisms properly, while investigating data access issues there is a chance of problems arising like data inconsistency and dead-lock.
- If many threads try to access the same data, then there is a chance that the situation of thread starvation may arise. Resource contention issues are another problem that can trouble the user.
- Display issues may occur if threads lack coordination when displaying data.

Benefits of Multithreading

Multithreading can improve the performance and efficiency of a program by utilizing the available CPU resources more effectively. Executing multiple threads concurrently, it can take advantage of parallelism and reduce overall execution time.

Multithreading can enhance responsiveness in applications that involve user interaction. By separating time-consuming tasks from the main thread, the user interface can remain responsive and not freeze or become unresponsive.

Multithreading can enable better resource utilization. For example, in a server application, multiple threads can handle incoming client requests simultaneously, allowing the server to serve more clients concurrently.

Multithreading can facilitate better code organization and modularity by dividing complex tasks into smaller, manageable units of execution. Each thread can handle a specific part of the task, making the code easier to understand and maintain.

Thread Models

Many-to-One:

- Many user threads mapped to one kernel thread.
- Limitation: one thread blocking blocks all.
- Example: Green threads (Java 1.x).

One-to-One:

- Each user thread maps to a kernel thread.
- Example: Windows, Linux (pthreads).

Many-to-Many:

- Many user threads mapped to many kernel threads.
- Combines flexibility & performance.

Synchronization Tools

Mutex Locks: Only one thread can access a resource at a time.

Semaphores: Counters to manage multiple accesses.

Monitors: High-level synchronization (e.g., Java synchronized blocks).

Condition Variables: Used for signaling between threads.

Deadlock in Operating System

A deadlock is a situation where a set of processes is blocked because each process is holding a resource and waiting for another resource acquired by some other process.

What is Deadlock?

Deadlock is a situation in computing where two or more processes are unable to proceed because each is waiting for the other to release resources. Key conditions for it to happen include mutual exclusion, resource holding, circular wait, and no preemption.



What is a Deadlock?

A situation where **two or more threads wait forever** for each other to release resources.

Example:

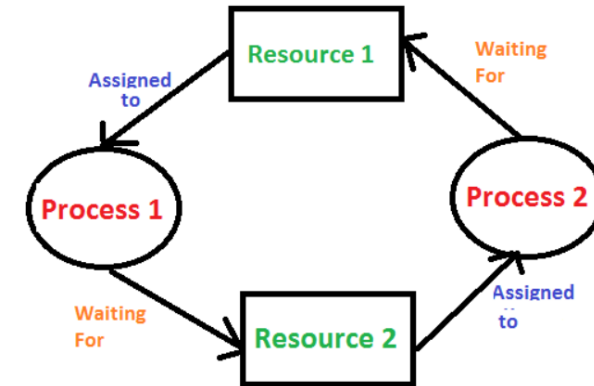
- Thread A holds Resource 1, needs Resource 2.
- Thread B holds Resource 2, needs Resource 1.
- Both wait forever.

How Does Deadlock occur in the Operating System?

Before going into detail about how deadlock occurs in the Operating System, let's first discuss how the Operating System uses the resources present. A process in an operating system uses resources in the following way.

- Requests a resource
- Use the resource
- Releases the resource

A situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by other(s). For example, in the aside diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



Necessary Conditions for Deadlock in OS

Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)

- **Mutual Exclusion:** Two or more resources are non-shareable (Only one process can use at a time).
- **Hold and Wait:** A process is holding at least one resource and waiting for resources.
- **No Preemption:** A resource cannot be taken from a process unless the process releases the resource.
- **Circular Wait:** A set of processes waiting for each other in circular form.

What is Deadlock Detection?

Deadlock detection is a process in computing where the system checks if there are any sets of processes that are stuck waiting for each other indefinitely, preventing them from moving forward. In simple words, deadlock detection is the process of finding out whether any process are stuck in loop or not.

What are the Methods For Handling Deadlock?

There are three ways to handle deadlock

- Deadlock Prevention or Avoidance
- Deadlock Recovery
- Deadlock Ignorance

1. Deadlock Prevention or Avoidance

Deadlock Prevention and Avoidance is the one of the methods for handling deadlock. First, we will discuss Deadlock Prevention, then Deadlock Avoidance.

- **What is Deadlock Prevention?**

In deadlock prevention the aim is to not let full-fill one of the required condition of the deadlock. This can be done by this method:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

(i) Mutual Exclusion

Locks are used (mutual exclusion) only for resources that can't be shared, like a printer, to ensure only one process uses them at a time. For resources that can be shared, such as read-only files, we don't need locks. This way, multiple processes can access shared resources simultaneously without issues.

The challenge is that mutual exclusion is required for non-sharable resources, while sharable resources can be accessed by multiple processes freely.

(ii) Hold and Wait

To prevent "hold and wait" in a system, the OS ensures that a process doesn't hold any resources while waiting for others.

This can be done by the OS providing a process with all the resources it needs right at the start. However, this can be inefficient. For example, if a process needs three resources—say a DVD drive, a file, and a printer—but only needs the printer an hour later, holding onto it the entire time can block other processes that need it. This could lead to "starvation," where other processes wait unnecessarily for a resource that could be used and released by them sooner.

Alternatively, the OS can release resources when they aren't in use. For instance, if a process first requests the DVD drive and a file to copy data, it uses them and releases them when done. Later, when it needs to print, it requests the printer without holding onto any other resources it's not actively using. This approach reduces waiting and makes resources more available to other processes.

(iii) No Preemption

In non-preemption, if a process is holding a resource and requests additional resources that are currently unavailable, the OS may preempt (or release) the resources the process is already holding. The process will then need to request both the old and new resources again to resume execution.

For example, if process P1 holds resource R1 and requests R2, which is currently held by process P2, the OS will preempt R1 from P1. Later, P1 will attempt to restart by requesting both R1 and R2.

Problem: This can lead to a *live lock*.

Live Lock: Live lock occurs when two or more processes continuously change their state in response to each other without making real progress.

(iv) Circular Wait

To prevent circular wait in a system, the OS can enforce an ordering of resources that processes must follow when requesting them. This means each process must acquire resources in a specific, pre-defined order, which prevents circular dependencies.

Example: If there are two processes, P1 and P2, and two resources, R1 and R2, the OS can set a rule that a process must acquire R1 before it can request R2. So, if P1 has acquired R1, only then can it request R2. Meanwhile, P2 will have to wait until R1 is free before it can proceed. By following this strict order, circular waiting is avoided, as no process can hold one resource while waiting indefinitely for another that another process holds.

This method is one way to prevent deadlock, specifically by addressing the circular wait condition. Although there are several deadlock prevention techniques, this ordering method is generally more practical and effective than others.

What is Deadlock Avoidance?

Proactively manages resources to prevent deadlock before it happens. It uses a **Futuristic Approach** that anticipates and evaluates resource allocation needs to avoid future deadlock scenarios.

Advance Resource Information:

- Requires knowledge of all resources a process will need before it begins execution.

Banker's Algorithm:

- Checks if resources can be safely allocated without causing deadlock.
- Acts as a "safety check" before granting resources.

Safety Check Process:

- Simulates allocation of resources to processes to ensure no unsafe states occur.
- System only proceeds with allocation if it's safe.

Performance Trade-off:

- Guarantees system stability and data correctness.
- Can reduce system efficiency due to resource-checking overhead.

What is Deadlock Recovery?

If Deadlock prevention or avoidance is not applied to the software then we can handle this by deadlock detection and recovery. which consist of two phases:

- In the first phase, OS examine the state of the process and check whether there is a deadlock or not in the system.
- If found deadlock in the first phase then OS apply the algorithm for recovery of the deadlock.

In Deadlock detection and recovery, OS get the correctness of data but performance decreases.

Methods of Deadlock Recovery

There are several Deadlock Recovery Techniques:

- Manual Intervention
- Automatic Recovery
- Process Termination
- Resource Preemption

1. Manual Intervention

- When a deadlock is detected, one option is to inform the operator and let them handle the situation manually. While this approach allows for human judgment and decision-making, it can be time-consuming and may not be feasible in large-scale systems.

2. Automatic Recovery

- An alternative approach is to enable the system to recover from deadlock automatically. This method involves breaking the deadlock cycle by either aborting processes or preempting resources.

3. Process Termination

Abort all Deadlocked Processes

- This approach breaks the deadlock cycle, but it comes at a significant cost. The processes that were aborted may have executed for a considerable amount of time, resulting in the loss of partial computations. These computations may need to be recomputed later.

Abort one process at a time

- Instead of aborting all deadlocked processes simultaneously, this strategy involves selectively aborting one process at a time until the deadlock cycle is eliminated. However, this incurs overhead as a deadlock-detection algorithm must be invoked after each process termination to determine if any processes are still deadlocked.

Factors for choosing the termination order:

The process's priority

Completion time and the progress made so far

Resources consumed by the process

Resources required to complete the process

Number of processes to be terminated

Process type (interactive or batch)

4. *Resource Preemption*

Selecting a Victim: Resource preemption involves choosing which resources and processes should be preempted to break the deadlock. The selection order aims to minimize the overall cost of recovery. Factors considered for victim selection may include the number of resources held by a deadlocked process and the amount of time the process has consumed.

Rollback: If a resource is preempted from a process, the process cannot continue its normal execution as it lacks the required resource. Rolling back the process to a safe state and restarting it is a common approach. Determining a safe state can be challenging, leading to the use of total rollback, where the process is aborted and restarted from scratch.

Starvation Prevention: To prevent resource starvation, it is essential to ensure that the same process is not always chosen as a victim. If victim selection is solely based on cost factors, one process might repeatedly lose its resources and never complete its designated task. To address this, it is advisable to limit the number of times a process can be chosen as a victim, including the number of rollbacks in the cost factor.

What is Deadlock Ignorance?

If a deadlock is very rare, then let it happen and reboot the system. This is the approach that both Windows and UNIX take. we use the ostrich algorithm for deadlock ignorance.

In Deadlock, ignorance performance is better than the above two methods but the correctness of data is not there.

Safe State

A safe state can be defined as a state in which there is no deadlock. It is achievable if:

- If a process needs an unavailable resource, it may wait until the same has been released by a process to which it has already been allocated. if such a sequence does not exist, it is an unsafe state.
- All the requested resources are allocated to the process.

Aspect	Deadlock	Starvation
Definition	A condition where two or more processes are blocked forever, each waiting for a resource held by another.	A condition where a process is perpetually denied necessary resources, despite resources being available.
Resource Availability	Resources are held by processes involved in the deadlock.	Resources are available but are continuously allocated to other processes.
Cause	Circular dependency between processes, where each process is waiting for a resource from another.	Continuous preference or priority given to other processes, causing a process to wait indefinitely.
Resolution	Requires intervention, such as aborting processes or preempting resources to break the cycle.	Can be mitigated by adjusting scheduling policies to ensure fair resource allocation.

Practical case in C Programming

- `#include <stdio.h>`
- `#include <windows.h>`
- `// Thread function - executed by each thread`
- `DWORD WINAPI MyThreadFunction(LPVOID lpParam) {`
- `int threadNum = *((int*)lpParam); // get thread number`
- `for (int i = 1; i <= 5; i++) {`
- `printf("Thread %d: Step %d\n", threadNum, i);`
- `Sleep(500); // pause 0.5 seconds to simulate work`
- `}`
- `printf("Thread %d: Finished!\n", threadNum);`
- `return 0;`
- `}`

- `int main() {`
- `HANDLE thread1, thread2;`
- `int t1 = 1, t2 = 2;`
- `printf("Main program: Starting threads...\n");`
- `// Create two threads`
- `thread1 = CreateThread(NULL, 0, MyThreadFunction, &t1, 0, NULL);`
- `thread2 = CreateThread(NULL, 0, MyThreadFunction, &t2, 0, NULL);`

- `if (thread1 == NULL || thread2 == NULL) {`
- `printf("Error creating threads!\n");`
- `return 1;`
- `}`
- `// Main thread also does something`
- `for (int i = 1; i <= 5; i++) {`
- `printf("Main program: Working step %d\n", i);`
- `Sleep(300);`
- `}`

- `// Wait until both threads finish`
- `WaitForMultipleObjects(2, (HANDLE[]){thread1, thread2}, TRUE, INFINITE);`
- `printf("Main program: All threads have finished.\n");`
- `// Close handles`
- `CloseHandle(thread1);`
- `CloseHandle(thread2);`
- `return 0;`
- `}`

How It Works

Step	Description
1	The main() function starts and prints a message.
2	Two threads are created using CreateThread() .
3	Each thread runs MyThreadFunction(), printing messages and sleeping for 0.5 seconds.
4	The main program also runs a loop while the threads are working.
5	The OS switches between threads — you'll see interleaved outputs .
6	WaitForMultipleObjects() ensures the main program waits until both threads finish.

Lab work for Deadlock

- Analyse the lab work sheet and do all exercises connected to them.

Group Project

Identify a **deadlock scenario** in an application (banking, database, transport system).

Propose a **detection and resolution strategy**.

Create a **flowchart + explanation** of how the OS manages the scenario.

Lesson 6. Memory Management

Thank you



For I know the thoughts that I think toward you, saith the LORD, thoughts of peace, and not of evil, to give you an expected end. Jeremy 29:11